

Tarea: Módulo local de validaciones y utilidades financieras

Objetivo

Crear un proyecto local e independiente llamado `smolbox-utils`.

El proyecto tendrá funciones reutilizables para:

- Validar la **estructura** de un RFC mexicano.
- Formatear cantidades como moneda mexicana.
- Convertir cantidades monetarias a texto para comprobantes.
- Generar datos de prueba ficticios.
- Mostrar y probar las funciones desde una interfaz sencilla en Streamlit.

Este proyecto **no se integrará todavía al sistema principal**. Debe funcionar por sí solo en tu computadora.

Alcance y reglas

Lo que sí debes hacer

- Usar **Python 3**.
- Crear el proyecto desde cero en tu computadora.
- Escribir el código tú mismo.
- Consultar documentación oficial de Python, Streamlit y material de referencia cuando sea necesario.
- Probar cada función con entradas válidas, inválidas y casos límite.
- Agregar comentarios breves solo donde ayuden a entender una decisión no obvia.
- Mantener el código ordenado y separado por responsabilidades.

Lo que no debes hacer

- Si usas IA, haz comentarios detallando cómo la utilizaste.

- Intenta no usar librerías externas para resolver directamente la conversión de números a letras; esa lógica debe ser implementada por ti.
- No usar datos fiscales reales para las pruebas.

Uso de asistentes de IA

Esta tarea busca que practiques lógica, lectura de documentación y depuración.

Si tienes una duda, primero intenta:

1. Leer el mensaje de error.
2. Revisar la documentación oficial.
3. Buscar una explicación técnica y entenderla.
4. Preguntar al equipo describiendo qué intentaste, qué esperabas y qué error obtuviste.

Al final deberás poder explicar con tus propias palabras cómo funciona cada función.

Estructura esperada

Crea una carpeta llamada:

```
smolbox-utils/
```

Dentro, crea la siguiente estructura:

```
smolbox-utils/  
|  
├─ app.py  
├─ utils.py  
├─ test_utils.py  
├─ requirements.txt  
└─ README.md
```

Propósito de cada archivo

Archivo	Contenido esperado
utils.py	Funciones principales del proyecto

app.py	Interfaz visual con Streamlit
test_utils.py	Pruebas automatizadas de las funciones
requirements.txt	Dependencias necesarias para ejecutar el proyecto
README.md	Instrucciones para instalar, ejecutar y probar el proyecto

Paso 1: Preparar el entorno

Desde una terminal, entra a la carpeta del proyecto y crea un entorno virtual:

```
python -m venv .venv
```

Activa el entorno virtual.

En Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

En Windows CMD:

```
.venv\Scripts\activate.bat
```

Instala Streamlit:

```
pip install streamlit
```

Guarda la dependencia en requirements.txt:

```
pip freeze > requirements.txt
```

Para ejecutar la aplicación al final, usarás:

```
streamlit run app.py
```

Si el comando anterior no funciona, prueba:

```
python -m streamlit run app.py
```

Paso 2: Implementar las utilidades

Todas las funciones deben estar dentro de `utils.py`.

Cada función debe:

- Tener un nombre claro.
- Recibir parámetros definidos.
- Devolver un resultado.
- No imprimir resultados directamente con `print()`.
- No depender de Streamlit.
- Manejar entradas inválidas sin detener todo el programa.

Puedes usar `print()` únicamente en pruebas temporales mientras desarrollas, pero el entregable final debe usar `return`.

Función 1: Validar estructura de RFC

Nombre

```
validar_rfc(rfc)
```

Objetivo

Validar si un texto cumple con la **estructura general** de un RFC mexicano para:

- Persona física: 13 caracteres.
- Persona moral: 12 caracteres.

Importante

Esta función **no valida que el RFC exista ante el SAT**. Solo revisa que tenga una estructura esperada: prefijo alfabético, fecha con seis dígitos y homoclave alfanumérica.

Requisitos

- Debe aceptar texto.
- Debe ignorar espacios al inicio y al final.
- Debe convertir el RFC a mayúsculas antes de validarlo.
- Debe usar una expresión regular (re).
- Debe devolver True si cumple el patrón.
- Debe devolver False si no cumple el patrón.
- Si recibe None, un número, una lista u otro tipo que no sea texto, debe devolver False.
- No debe lanzar una excepción por una entrada inválida.

Casos que debe aceptar estructuralmente

```
"ABCD010101A1B" # Persona física: 13 caracteres
"ABC010101A1B"  # Persona moral: 12 caracteres
" abcd010101a1b " # Debe aceptar espacios exteriores y minúsculas
```

Casos que debe rechazar

```
"RFC_INVALIDO"
"ABC123"
"ABCD010101A1B2"
"ABC010101@@@"
123456789
None
""
```

Criterio de aceptación

La función debe reconocer ambas longitudes esperadas y rechazar entradas con caracteres no permitidos o longitud incorrecta.

Función 2: Formatear moneda mexicana

Nombre

```
formatear_moneda(monto)
```

Objetivo

Recibir un monto numérico y devolver una cadena con formato monetario mexicano.

Ejemplos esperados

```
formatear_moneda(1250.5)
# "$1,250.50"
```

```
formatear_moneda(3000)
# "$3,000.00"
```

```
formatear_moneda(0)
# "$0.00"
```

```
formatear_moneda(-10.25)
# "-$10.25"
```

Requisitos

- Debe aceptar números enteros y decimales.
- Debe redondear a dos decimales.
- Debe incluir separador de miles.
- Debe incluir el símbolo \$.
- Debe conservar el signo negativo, si existe.
- No debe aceptar valores booleanos como `True` o `False`, aunque Python los trate internamente como números.
- Si recibe una entrada inválida, debe lanzar un `ValueError` con un mensaje claro.

Entradas inválidas de ejemplo

```
"1250.50"
```

```
None
```

```
True
```

```
[100, 200]
```

Mensaje esperado para error

No es necesario que sea idéntico, pero debe ser claro. Ejemplo:

```
El monto debe ser un número entero o decimal.
```

Función 3: Convertir monto a letras

Nombre

```
numero_a_letras(monto)
```

Objetivo

Convertir una cantidad monetaria a una representación textual en pesos mexicanos, usando el formato:

```
LETRAS PESOS XX/100 M.N.
```

Ejemplos esperados

```
numero_a_letras(0)
```

```
# "CERO PESOS 00/100 M.N."
```

```
numero_a_letras(1)
```

```
# "UN PESO 00/100 M.N."
```

```
numero_a_letras(21.5)
```

```
# "VEINTIÚN PESOS 50/100 M.N."
```

```
numero_a_letras(150.50)
```

```
# "CIENTO CINCUENTA PESOS 50/100 M.N."
```

```
numero_a_letras(1000)
```

```
# "MIL PESOS 00/100 M.N."
```

```
numero_a_letras(2001.25)
```

```
# "DOS MIL UN PESOS 25/100 M.N."
```

Alcance obligatorio

La función debe soportar cantidades desde:

0.00 hasta 999,999.99

Reglas

- La parte entera se convierte a letras.
- La parte decimal se representa como dos dígitos sobre 100.
- El resultado debe quedar en mayúsculas.
- Debe usar PESO en singular para 1.
- Debe usar PESOS en plural para cualquier otro valor.
- Debe redondear a dos decimales antes de convertir.
- No debe usar librerías de terceros que conviertan números a texto.
- Para evitar errores de precisión con decimales, investiga y usa Decimal del módulo estándar decimal de Python.

Casos a considerar

0

1

1.01

10

16

21

29

30

31

99

100

101

```
115
121
200
500
999
1000
1001
1100
2000
100000
999999.99
```

Entradas inválidas

- Valores negativos.
- Valores mayores a 999999.99.
- Texto.
- None.
- Booleanos.

En estos casos debe lanzar `ValueError` con un mensaje claro.

Sugerencia de diseño

No intentes resolver toda la función en un bloque muy largo. Divide la lógica en funciones auxiliares privadas, por ejemplo:

```
_convertir_unidades(numero)
_convertir_decenas(numero)
_convertir_centenas(numero)
_convertir_miles(numero)
```

Las funciones privadas pueden iniciar con guion bajo, por ejemplo:

```
def _convertir_centenas(numero):
    ...
```

Paso 3: Crear pruebas automatizadas

Crea el archivo:

```
test_utils.py
```

Usa el módulo estándar `unittest` de Python. No es necesario instalar una librería adicional.

Debes incluir al menos:

Función	Casos mínimos
<code>validar_rfc()</code>	3 pruebas
<code>formatear_moneda()</code>	3 pruebas
<code>numero_a_letras()</code>	3 pruebas
<code>generar_comprobante_mock()</code>	3 pruebas

En total debes tener **al menos 12 pruebas**.

Ejemplos de tipos de prueba

- Entrada correcta.
- Entrada incorrecta.
- Valor vacío.
- Valor límite.
- Tipo de dato no esperado.
- Número negativo.
- Redondeo de decimales.
- Singular y plural: UN PESO frente a PESOS.
- RFC con espacios y minúsculas.
- RFC con longitud inválida.
- Monto máximo permitido.

- Monto mayor al límite.

Ejecución de pruebas

Las pruebas deben ejecutarse con:

```
python -m unittest test_utils.py
```

El resultado final debe indicar que todas las pruebas pasan.

Paso 4: Crear interfaz en Streamlit

Crea el archivo:

```
app.py
```

La interfaz debe importar las funciones desde `utils.py`. No debes duplicar la lógica dentro de `app.py`.

Ejemplo de importación:

```
from utils import (  
    validar_rfc,  
    formatear_moneda,  
    numero_a_letras  
)
```

Requisitos visuales mínimos

La página debe incluir:

- Título: Smolbox Utils.
- Breve descripción de la aplicación.
- Cuatro secciones, pestañas o expansores:
 - Validar RFC.
 - Formatear moneda.
 - Convertir número a letras.

- Campos de captura adecuados para cada función.
- Un botón para ejecutar cada acción.
- Resultado visible y comprensible.
- Mensajes de error claros cuando el usuario ingrese datos inválidos.

Comportamiento esperado

Sección “Validar RFC”

- Campo de texto para capturar RFC.
- Botón Validar RFC.
- Si cumple la estructura, mostrar mensaje de éxito.
- Si no cumple, mostrar mensaje de error.
- Debe aclarar en pantalla:

Esta validación solo revisa el formato estructural; no confirma que el RFC exista ante el SAT.

Sección “Formatear moneda”

- Campo numérico para introducir un monto.
- Botón Formatear.
- Mostrar resultado como texto.

Sección “Convertir número a letras”

- Campo numérico para monto.
- Botón Convertir a letras.
- Mostrar el resultado completo.

Paso 5: Documentar el proyecto

Crea un archivo README.md con estas secciones:

```
# Smolbox Utils
```

```
## Descripción
```

```
## Requisitos
```

```
## Instalación
```

```
## Ejecución de la aplicación
```

```
## Ejecución de pruebas
```

```
## Funciones disponibles
```

```
## Decisiones técnicas
```

```
## Limitaciones
```

Contenido mínimo del README

Debe explicar:

- Qué hace el proyecto.
- Qué versión de Python utilizaste.
- Cómo crear y activar el entorno virtual.
- Cómo instalar dependencias.
- Cómo ejecutar la aplicación Streamlit.
- Cómo ejecutar las pruebas.
- Qué hace cada función.
- Que `validar_rfc()` solo valida estructura y no autenticidad fiscal.
- Que los datos son ficticios.
- Una breve explicación de por qué usaste `Decimal` para los montos.
- Una breve explicación de la expresión regular utilizada para RFC.

Entregable

Entrega una carpeta comprimida con nombre:

smolbox-utils-tu_nombre.zip

Antes de comprimirla:

1. Elimina la carpeta `.venv`.
2. Elimina carpetas como `__pycache__`.
3. Verifica que no haya datos reales ni credenciales.
4. Verifica que `requirements.txt` y `README.md` estén incluidos.
5. Ejecuta las pruebas una última vez.
6. Ejecuta la aplicación una última vez.
7. Link de streamlit (despliegue).

Demostración final

1. La estructura de carpetas.
2. El contenido general de `utils.py`.
3. La explicación de la regex de RFC.
4. Dos pruebas correctas y dos incorrectas para RFC.
5. Un ejemplo de moneda con miles y decimales.
6. Un ejemplo de número a letras.
7. La generación de un comprobante ficticio.
8. La ejecución de las pruebas automatizadas.
9. La aplicación funcionando en Streamlit.
10. Una explicación de una dificultad que encontraste y cómo la resolviste.

Checklist antes de entregar

- ☐ Creé el proyecto con la estructura solicitada.
- ☐ Implementé las cuatro funciones en `utils.py`.
- ☐ Validé entradas correctas, incorrectas y casos límite.
- ☐ Implementé al menos 30 pruebas automatizadas.
- ☐ Todas las pruebas pasan correctamente.
- ☐ Mi aplicación de Streamlit abre y funciona.
- ☐ Documenté instalación, uso y limitaciones en el README.
- ☐ No incluí datos reales, credenciales ni archivos de entorno virtual.
- ☐ Puedo explicar el funcionamiento de mi código y mis decisiones técnicas.